ORACLE

Java Professional

Annotations

ITVDN
IT VIDEO DEVELOPERS NETWORK

Інформаційний відеосервіс для розробників програмного забезпечення

Автор курсу

Євген Тихонов

Після уроку обов’язково

Повторіть цей урок у форматі відео на ITVDN.com

Доступ можна отримати через керівництво вашого навчального центру

Перевірте, як Ви засвоїли цей матеріал на TestProvider.com

Annotations

Анотації

Annotation

Анотації – метадані, які можуть додаватися у вихідний код програми та семантично не впливають на неї, але можуть використовуватися в процесі аналізу коду, компіляції та навіть під час виконання.

Основні варіанти використання анотацій:

- надавати потрібну інформацію для компілятора;
- надавати метадані різним інструментам для генерації коду, конфігурацій тощо;
- використовуватись у коді під час виконання програмного коду (reflection).

Вбудовані анотації

Integrated Annotations

@Retention – це анотація, яка призначена для застосування лише як анотації до інших анотацій. Визначає політику утримання.

@Documented – це маркер-інтерфейс, який повідомляє інструменту, що анотація має бути документована.

@Target – це анотація, що визначає тип оголошення, до якого може бути застосована анотація. Приймає один аргумент, який має бути константою з перерахунку ElementType.

Наприклад, щоб вказати, що анотація застосовується лише до полів і локальних змінних:

@Target({ ElementType.FIELD, ElementTyle.LOCAL_VARIABLE})

@Inherited – це анотація-маркер, яка може застосовуватися в іншому оголошенні анотації. Вона стосується лише тих анотацій, які будуть використані в оголошеннях класів. Ця анотація дає змогу анотації суперкласу бути успадкованою в підкласі.

Вбудовані анотації

Integrated Annotations

@Override – анотація-маркер, яка може застосовуватися лише до методів. Метод, анотований як @Override, має перевизначати метод суперкласу.

@Deprecated – анотація, яка вказує на те, що оголошення застаріло та має бути замінено на нову форму.

@SafeVarargs – анотація-маркер, яка застосовується до методів і конструкторів. Вона вказує на те, що ніякі небезпечні дії, які пов'язані з параметром змінної кількості аргументів, неприпустимі.

Застосовується лише до методів і конструкторів зі змінною кількістю аргументів, які оголошені як static чи final.

@SuppressWarnings – ця анотація вказує, що одне або більше попереджень, які можуть бути видані компілятором, варто придушити.

Створення власних анотацій

Creating own annotation

Анотації створюються з використанням механізму, який ґрунтується на інтерфейсі.

Приклад оголошення анотації:

```
@interface My {
    String str();
    int val();
}
```

Символ @ вказує компілятору, що оголошена анотація. Методи, які оголошені в анотації, поводяться як поля.

Анотування методу :

```
@My (str = "Приклад анотації", val = 123)
public static void myMethod() { // ...
```

Політика утримання анотацій

Retention Policy

Є 3 політики утримання анотацій.

Політика утримання визначає, в якій точці анотація скидається:

- RetentionPolicy.SOURCE – анотація використовується на етапі компіляції та має відкидатися компілятором;
- RetentionPolicy.CLASS – анотація буде записана в class-файл компілятором, але не має бути доступною під час виконання (runtime). За замовчуванням у всіх анотацій .class.
- RetentionPolicy.RUNTIME – анотація буде записана в class-файл і доступна під час виконання через reflection.

Отримання анотації

Getting annotations

Рефлексія – цей засіб дозволяє отримати інформацію про клас під час виконання програми.

```
Class<?> c = ob.getClass();
Method m = c.getMethod("myMeth");
My anno = m.getAnnotation(My.class);
```

Використовуючи рефлексію, ми отримуємо інструкцію MyAnno, асоційовану з методом myMeth().

Документація

Documented

1. Використовуючи анотацію @Documented, ми говоримо, що анотація буде задокументована у JavaDoc.
2. Використати документаційні коментарі:

```
/**
 * Class created by Yevhenii
 * @param
 * @author
 */
```

Щоб згенерувати JavaDoc в IDEA, необхідно:

Натиснути Tools - > Generate JavaDoc – > вказати шлях, куди збережемо документацію

Дивіться наші уроки у відеоформаті

ITVDN.com

Перегляньте цей урок у відеоформаті на освітньому порталі ITVDN.com для закріплення пройденого матеріалу.

Усі курси записані сертифікованими тренерами, які працюють у навчальному центрі CyberBionic Systematics.

Перевірка знань

TestProvider.com

TestProvider – це online-сервіс перевірки знань з інформаційних технологій. Завдяки йому Ви можете оцінити Ваш рівень та виявити слабкі місця. Він буде корисним як у процесі вивчення технології, так і загальної оцінки знань IT-фахівця.

Після кожного уроку проходьте тестування для перевірки знань на TestProvider.com.

Успішне проходження фінального тестування дасть Вам змогу отримати відповідний Сертифікат.

Q&A

ITVDN
IT VIDEO DEVELOPERS NETWORK